```
import os
import time
import pyotp
import pandas as pd
import requests
import datetime
import pytz
import threading
import asyncio
import numpy as np
import json
import tempfile
from dotenv import load_dotenv
from SmartApi import SmartConnect
from SmartApi.smartWebSocketV2 import SmartWebSocketV2

# Load Environment Variables
load_dotenv()
API_KEY = os.getenv('ANGEL_API_KEY')
CLIENT_ID = os.getenv('CLIENT_ID')
PASSWORD = os.getenv('PASSWORD')
TOTP_SECRET_KEY = os.getenv('TOTP_SECRET_KEY')
TELEGRAM_BOT_TOKEN = os.getenv('TELEGRAM_BOT_TOKEN')
TELEGRAM_CHAT_ID = os.getenv('TELEGRAM_CHAT_ID')

IST = pytz.timezone('Asia/Kolkata')

# Constants & Risk Limits
INDEX_TOKEN = "99926000"
NIFTY_LOT_SIZE = 65
LOTS = 5
TOTAL_QTY = NIFTY_LOT_SIZE * LOTS

MAX_DAILY_LOSS = -6000.0
MAX_DAILY_TRADES = 6
SLIPPAGE = 0.50
FRICTION_PER_LOT = 9.0  # Approx ₹9 per lot per round-trip for STT/Exchange fees

# Globals
live_prices = {}
ws_instance = None
last_tick_time = time.time()
telegram_urgent = asyncio.Queue()
telegram_batch = asyncio.Queue()
```

```
MAIN_LOOP = None
is_reconnecting = False

# ==================== ASYNC TELEGRAM DISPATCHER ====================
async def telegram_worker():
    batch_buffer = []
    last_batch_time = time.time()

    async def send_to_tg(text):
        try:
            url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
            payload = {"chat_id": TELEGRAM_CHAT_ID, "text": text, "parse_mode": "HTML"}
            await asyncio.to_thread(requests.post, url, json=payload, timeout=5)
        except Exception as e:
            print(f"Telegram Alert Failed: {e}")

    while True:
        # 1. Dispatch Urgent Messages Immediately
        try:
            msg = telegram_urgent.get_nowait()
            timestamp = datetime.datetime.now(IST).strftime("%H:%M:%S")
            await send_to_tg(f"[{timestamp}] {msg}")
            await asyncio.sleep(0.5)
            continue
        except asyncio.QueueEmpty:
            pass

        # 2. Buffer Low-Priority Messages
        try:
            msg = telegram_batch.get_nowait()
            batch_buffer.append(msg)
        except asyncio.QueueEmpty:
            pass

        # 3. Flush Batched Messages
        if batch_buffer and (time.time() - last_batch_time > 3.0 or len(batch_buffer) >= 4):
            timestamp = datetime.datetime.now(IST).strftime("%H:%M:%S")
            combined_msg = "\n".join(batch_buffer)
            await send_to_tg(f"[{timestamp}] 📋 <b>Digest:</b>\n{combined_msg}")
            batch_buffer.clear()
            last_batch_time = time.time()
            await asyncio.sleep(0.5)

        await asyncio.sleep(0.1)
```

```
def send_alert(message, urgent=True):
    global MAIN_LOOP
    if MAIN_LOOP and MAIN_LOOP.is_running():
        queue = telegram_urgent if urgent else telegram_batch
        MAIN_LOOP.call_soon_threadsafe(queue.put_nowait, message)

# ==================== WEBSOCKET HANDLERS ====================
def on_data(wsapp, message):
    global last_tick_time
    try:
        token = str(message.get("token"))
        ltp = message.get("last_traded_price", 0)
        vol = message.get("volume_traded_today", 0)
        if token and ltp:
            price = float(ltp) / 100 if isinstance(ltp, int) else float(ltp)
            live_prices[token] = {"ltp": price, "vol": vol, "time": time.time()}
            last_tick_time = time.time()
    except Exception:
        pass

def on_open(wsapp):
    send_alert("🟢 <b>WebSocket:</b> Connection Opened.", urgent=False)

def on_error(wsapp, error):
    send_alert(f"🔴 <b>WebSocket Error:</b> {error}", urgent=True)

def on_close(wsapp):
    send_alert("🔴 <b>WebSocket:</b> Connection Closed.", urgent=False)

def subscribe_tokens(ce_token, pe_token):
    global ws_instance
    if not ws_instance:
        return
    token_list = [{"exchangeType": 1, "tokens": [INDEX_TOKEN]}]
    opt_tokens = [str(t) for t in [ce_token, pe_token] if t]
    if opt_tokens:
        token_list.append({"exchangeType": 2, "tokens": opt_tokens})
    ws_instance.subscribe("token_subscription", 1, token_list)
    send_alert("📡 Subscribed to market feeds.", urgent=False)

def unsubscribe_tokens(ce_token, pe_token):
    global ws_instance
    if not ws_instance or not ce_token or not pe_token:
```

```
        return
    token_list = [{"exchangeType": 2, "tokens": [str(ce_token), str(pe_token)]}]
    try:
        ws_instance.unsubscribe("token_subscription", 1, token_list)
    except Exception:
        pass
    live_prices.pop(str(ce_token), None)
    live_prices.pop(str(pe_token), None)

# ==================== STRIKE SELECTOR ====================
class StrikeSelector:
    def __init__(self, api_client):
        self.api = api_client
        self.scrip_df = pd.DataFrame()
        self.load_scrip_master()

    def load_scrip_master(self):
        today_str = datetime.datetime.now(IST).strftime("%Y%m%d")
        cache_file = f"scrip_master_{today_str}.json"
        if os.path.exists(cache_file):
            try:
                self.scrip_df = pd.read_json(cache_file)
                self.scrip_df['expiry_date'] = pd.to_datetime(self.scrip_df['expiry_date'])
                return
            except Exception:
                pass

        try:
            send_alert("📥 Downloading Scrip Master...", urgent=False)
            response =
requests.get("https://margincalculator.angelbroking.com/OpenAPI_File/files/OpenAPIScripMaste
r.json", timeout=15)
            df = pd.DataFrame(response.json())
            df = df[(df['name'] == 'NIFTY') & (df['exch_seg'] == 'NFO') & (df['instrumenttype'] ==
'OPTIDX')].copy()
            df['strike_num'] = (pd.to_numeric(df['strike'], errors='coerce') / 100).round(2)
            df['expiry_date'] = pd.to_datetime(df['expiry'])
            self.scrip_df = df
            self.scrip_df.to_json(cache_file)
        except Exception as e:
            send_alert(f"🔴 Scrip Master Error: {e}", urgent=True)

    def get_tokens(self, current_index_ltp):
        if self.scrip_df.empty or not current_index_ltp:
```

```
            return None, None, None, None, None
        atm_strike = round(current_index_ltp / 50) * 50
        now = datetime.datetime.now(IST)
        future_expiries = self.scrip_df[self.scrip_df['expiry_date'] >= pd.Timestamp(now.date())]
        if future_expiries.empty:
            return None, None, None, None, None
        nearest_expiry = future_expiries['expiry_date'].min()
        active_contracts = future_expiries[future_expiries['expiry_date'] == nearest_expiry]

        # EXPIRY REGIME
        if nearest_expiry.date() == now.date() and now.time() >= datetime.time(12, 30):
            ce_match = active_contracts[(active_contracts['strike_num'] == atm_strike) &
(active_contracts['symbol'].str.endswith('CE'))]
            pe_match = active_contracts[(active_contracts['strike_num'] == atm_strike) &
(active_contracts['symbol'].str.endswith('PE'))]
            if not ce_match.empty and not pe_match.empty:
                return atm_strike, ce_match.iloc[0]['symbol'], str(ce_match.iloc[0]['token']),
pe_match.iloc[0]['symbol'], str(pe_match.iloc[0]['token'])

        # STANDARD REGIME
        range_contracts = active_contracts[(active_contracts['strike_num'] >= atm_strike - 400) &
(active_contracts['strike_num'] <= atm_strike + 400)]
        ce_tokens =
range_contracts[range_contracts['symbol'].str.endswith('CE')]['token'].astype(str).tolist()
        pe_tokens =
range_contracts[range_contracts['symbol'].str.endswith('PE')]['token'].astype(str).tolist()

        try:
            res = self.api.getMarketData(mode="LTP", exchangeTokens={"NFO": ce_tokens +
pe_tokens})
            market_data = res.get('data', {}).get('fetched', []) if isinstance(res, dict) else []
        except Exception:
            market_data = []

        best_ce, best_pe = None, None
        min_ce_diff, min_pe_diff = float('inf'), float('inf')

        for item in market_data:
            token = str(item.get('exchangeToken'))
            ltp = item.get('ltp', 0)
            if ltp == 0: continue
            diff = abs(ltp - 100)
            if token in ce_tokens and diff < min_ce_diff:
                min_ce_diff, best_ce = diff, token
```

```
            elif token in pe_tokens and diff < min_pe_diff:
                min_pe_diff, best_pe = diff, token

        if best_ce and best_pe:
            ce_row = range_contracts[range_contracts['token'].astype(str) == best_ce].iloc[0]
            pe_row = range_contracts[range_contracts['token'].astype(str) == best_pe].iloc[0]
            return atm_strike, ce_row['symbol'], str(ce_row['token']), pe_row['symbol'],
str(pe_row['token'])

        return None, None, None, None, None

# ==================== O(1) STREAMING INDICATORS ====================
class HybridIndicators:
    def __init__(self):
        self.ema9 = None
        self.ema15 = None
        self.vwap = 0.0
        self.cum_vol = 0
        self.cum_pv = 0.0
        self.prev_close = None
        self.atr14 = 0.0
        self.vol_profile = {}
        self.last_minute = None
        self.candle_high = 0.0
        self.candle_low = 0.0
        self.candle_close = 0.0

    def seed_from_rest(self, df):
        if df.empty or len(df) < 15:
            return
        self.ema9 = df['close'].ewm(span=9, adjust=False).mean().iloc[-1]
        self.ema15 = df['close'].ewm(span=15, adjust=False).mean().iloc[-1]
        high_low = df['high'] - df['low']
        high_close = np.abs(df['high'] - df['close'].shift())
        low_close = np.abs(df['low'] - df['close'].shift())
        ranges = pd.concat([high_low, high_close, low_close], axis=1)
        true_range = np.max(ranges, axis=1)
        self.atr14 = true_range.rolling(14).mean().iloc[-1]
        self.prev_close = df['close'].iloc[-1]

        recent_30 = df.tail(30)
        self.vol_profile = {}
        for _, row in recent_30.iterrows():
            price_level = round(row['close'])
```

```
            self.vol_profile[price_level] = self.vol_profile.get(price_level, 0) + row['volume']

    def get_poc(self):
        if not self.vol_profile:
            return None
        return max(self.vol_profile, key=self.vol_profile.get)

    def update_tick(self, price, volume, tick_timestamp):
        if volume > self.cum_vol:
            tick_vol = volume - self.cum_vol
            self.cum_vol = volume
            self.cum_pv += (price * tick_vol)
            self.vwap = self.cum_pv / self.cum_vol if self.cum_vol > 0 else price

        tick_dt = datetime.datetime.fromtimestamp(tick_timestamp, tz=IST)
        current_min = tick_dt.minute

        if self.last_minute is None:
            self.last_minute = current_min
            self.candle_high = self.candle_low = self.candle_close = price

        if price > self.candle_high:
            self.candle_high = price
        if price < self.candle_low:
            self.candle_low = price
        self.candle_close = price

        if current_min != self.last_minute:
            if self.ema9 is not None:
                self.ema9 = (self.candle_close - self.ema9) * (2 / 10) + self.ema9
                self.ema15 = (self.candle_close - self.ema15) * (2 / 16) + self.ema15
            tr = max(
                self.candle_high - self.candle_low,
                abs(self.candle_high - self.prev_close) if self.prev_close else 0,
                abs(self.candle_low - self.prev_close) if self.prev_close else 0
            )
            self.atr14 = ((self.atr14 * 13) + tr) / 14
            self.prev_close = self.candle_close
            self.last_minute = current_min
            self.candle_high = self.candle_low = price

# ==================== EXECUTION ENGINE ====================
class ScalpingEngine:
    def __init__(self, api_client):
```

```
        self.api = api_client
        self.idx_ind = HybridIndicators()
        self.ce_ind = HybridIndicators()
        self.pe_ind = HybridIndicators()
        self.active_tokens = {"ce_token": None, "ce_sym": None, "pe_token": None, "pe_sym":
None, "atm": 0}

        self.trade_state = {
            "date": "", "active": False, "type": None, "token": None, "sym": None,
            "entry_price": 0.0, "entry_idx_price": 0.0, "qty": TOTAL_QTY,
            "hard_sl": 0.0, "idx_sl": 0.0, "trailing_sl": 0.0,
            "target_1_hit": False, "entry_opt_atr": 0.0,
            "daily_pnl": 0.0, "trades_today": 0
        }
        self.trade_log = []
        self.load_state()

    def load_state(self):
        today_str = datetime.datetime.now(IST).strftime("%Y-%m-%d")
        if os.path.exists("trade_state.json"):
            try:
                with open("trade_state.json", "r") as f:
                    saved = json.load(f)
                    if saved.get("date") == today_str:
                        self.trade_state.update(saved)
                        if self.trade_state["active"]:
                            send_alert(f"🔄 <b>Recovered Active Trade:</b> {self.trade_state['sym']}",
urgent=True)
                    else:
                        send_alert("🧹 Stale state file found. Resetting daily metrics.", urgent=False)
            except Exception:
                pass

    def save_state(self):
        self.trade_state["date"] = datetime.datetime.now(IST).strftime("%Y-%m-%d")
        try:
            with tempfile.NamedTemporaryFile("w", dir=".", delete=False) as tf:
                json.dump(self.trade_state, tf, indent=2)
                temp_name = tf.name
            os.replace(temp_name, "trade_state.json")
        except Exception:
            pass

    def fetch_rest_seed(self, token, exchange="NFO"):
```

```
        now = datetime.datetime.now(IST)
        from_time = (now - datetime.timedelta(minutes=60)).strftime("%Y-%m-%d %H:%M")
        to_time = now.strftime("%Y-%m-%d %H:%M")
        try:
            res = self.api.getCandleData({
                "exchange": exchange, "symboltoken": str(token), "interval": "ONE_MINUTE",
                "fromdate": from_time, "todate": to_time
            })
            if res and res.get('status') and res.get('data'):
                df = pd.DataFrame(res['data'], columns=['ts', 'open', 'high', 'low', 'close', 'volume'])
                return df.apply(pd.to_numeric, errors='ignore')
        except Exception:
            pass
        return pd.DataFrame()

    async def execute_trade(self, opt_type, token, sym, raw_price, idx_price, atr_opt, atr_idx):
        if not atr_opt or atr_opt <= 0: atr_opt = 2.0
        if not atr_idx or atr_idx <= 0: atr_idx = 5.0

        entry_price = raw_price + SLIPPAGE  # Model Ask-side Slippage

        self.trade_state.update({
            "active": True, "type": opt_type, "token": str(token), "sym": sym,
            "entry_price": entry_price, "entry_idx_price": idx_price, "qty": TOTAL_QTY,
            "entry_opt_atr": atr_opt,
            "hard_sl": entry_price - (1.5 * atr_opt),
            "trailing_sl": entry_price - (1.5 * atr_opt),
            "idx_sl": idx_price - (1.0 * atr_idx) if opt_type == 'CE' else idx_price + (1.0 * atr_idx),
            "target_1_hit": False
        })
        self.trade_state["trades_today"] += 1
        self.save_state()

        send_alert(f"🚀 <b>PAPER ENTRY ({opt_type}):</b> {sym}\nEntry: ₹{entry_price:.2f} (Slip
Inc.)\nCapital SL: ₹{self.trade_state['hard_sl']:.2f}", urgent=True)

    async def exit_trade(self, raw_price, reason):
        exit_price = raw_price - SLIPPAGE  # Model Bid-side Slippage
        qty = self.trade_state['qty']

        gross_pnl = (exit_price - self.trade_state['entry_price']) * qty
        friction_cost = FRICTION_PER_LOT * (qty / NIFTY_LOT_SIZE)
        net_pnl = gross_pnl - friction_cost
```

```
        self.trade_state['daily_pnl'] += net_pnl
        self.trade_log.append({
            "sym": self.trade_state['sym'], "entry": self.trade_state['entry_price'],
            "exit": exit_price, "pnl": net_pnl, "reason": reason, "qty": qty
        })

        self.trade_state["active"] = False
        self.trade_state["qty"] = TOTAL_QTY  # Reset for next trade
        self.save_state()

        send_alert(f"🛑 <b>EXIT ({reason}):</b> {self.trade_state['sym']}\nExit: ₹{exit_price:.2f} |
Net PnL: {net_pnl:+.2f}", urgent=True)

    async def check_exits(self, raw_price, idx_price):
        if not self.trade_state["active"]: return
        ts = self.trade_state
        atr = ts["entry_opt_atr"]
        eval_price = raw_price - SLIPPAGE

        # 1. Structural Stop (Index)
        if (ts['type'] == 'CE' and idx_price < ts['idx_sl']) or (ts['type'] == 'PE' and idx_price >
ts['idx_sl']):
            await self.exit_trade(raw_price, "Structural Index Stop")
            return

        # 2. Capital Hard Stop / Trailing Stop
        if eval_price <= ts['trailing_sl']:
            await self.exit_trade(raw_price, "Trailing/Hard SL Hit")
            return

        # 3. Parabolic Cap
        if eval_price >= ts['entry_price'] + (3.5 * atr):
            await self.exit_trade(raw_price, "Parabolic Target (3.5 ATR)")
            return

        # 4. Partial Exit & Cost-Trailing
        if not ts['target_1_hit'] and eval_price >= ts['entry_price'] + (2.0 * atr):
            ts['target_1_hit'] = True
            ts['trailing_sl'] = ts['entry_price'] + (1.2 * atr)

            partial_qty = 3 * NIFTY_LOT_SIZE
            ts['qty'] -= partial_qty

            gross_pnl = (eval_price - ts['entry_price']) * partial_qty
```

```
            friction_cost = FRICTION_PER_LOT * (partial_qty / NIFTY_LOT_SIZE)
            net_pnl = gross_pnl - friction_cost

            self.trade_state['daily_pnl'] += net_pnl
            self.trade_log.append({
                "sym": ts['sym'], "entry": ts['entry_price'], "exit": eval_price,
                "pnl": net_pnl, "reason": "Partial Profit Booked", "qty": partial_qty
            })
            self.save_state()
            send_alert(f"💰 <b>PARTIAL BOOKED (2.0 ATR):</b> {ts['sym']}\nNet PnL:
{net_pnl:+.2f}\nTrail SL: ₹{ts['trailing_sl']:.2f}", urgent=True)

        elif not ts['target_1_hit'] and eval_price >= ts['entry_price'] + (1.0 * atr):
            if ts['trailing_sl'] < ts['entry_price']:
                ts['trailing_sl'] = ts['entry_price']
                self.save_state()
                send_alert("🛡️ <b>SL Trailed to Cost (1.0 ATR)</b>", urgent=False)

# ==================== MAIN LIFECYCLE ====================
async def main_loop():
    global ws_instance, last_tick_time, MAIN_LOOP, is_reconnecting
    MAIN_LOOP = asyncio.get_running_loop()
    asyncio.create_task(telegram_worker())

    api = None
    engine = None
    selector = None
    session_data = None

    while True:
        now = datetime.datetime.now(IST)
        current_time = now.time()

        # 1. Weekend / Night Sleep
        if now.weekday() >= 5 or current_time >= datetime.time(15, 30) or current_time <
datetime.time(8, 30):
            if ws_instance:
                try: ws_instance.close()
                except Exception: pass
                ws_instance = None
                send_alert("💤 <b>System Sleeping.</b>", urgent=False)
            await asyncio.sleep(60)
            continue
```

```
        # 2. Pre-Market Boot & Recovery (08:30 - 09:15)
        if not ws_instance and datetime.time(8, 30) <= current_time < datetime.time(9, 15):
            send_alert("⚙️ <b>Pre-Market Boot:</b> Authenticating...", urgent=True)
            api = SmartConnect(api_key=API_KEY)
            totp = pyotp.TOTP(TOTP_SECRET_KEY).now()
            session = api.generateSession(CLIENT_ID, PASSWORD, totp)

            if session.get('status'):
                session_data = session['data']
                engine = ScalpingEngine(api)
                selector = StrikeSelector(api)

                ws_instance = SmartWebSocketV2(session_data['jwtToken'], API_KEY, CLIENT_ID,
api.getfeedToken())
                ws_instance.on_data = on_data
                ws_instance.on_open = on_open
                ws_instance.on_error = on_error
                ws_instance.on_close = on_close
                threading.Thread(target=ws_instance.connect, daemon=True).start()

                if engine.trade_state["active"]:
                    subscribe_tokens(engine.trade_state["token"], None)
                else:
                    subscribe_tokens(None, None)
            await asyncio.sleep(60)
            continue

        # 3. Dynamic Strike Tracking (Locked During Active Trades)
        if engine and selector and datetime.time(9, 15) <= current_time < datetime.time(15, 15):
            idx_data = live_prices.get(INDEX_TOKEN)
            if idx_data and not engine.trade_state['active']:
                idx_ltp = idx_data['ltp']
                anchor = engine.active_tokens['atm']

                if not anchor or abs(idx_ltp - anchor) >= 200:
                    atm, ce_sym, ce_token, pe_sym, pe_token = selector.get_tokens(idx_ltp)
                    if ce_token and pe_token:
                        unsubscribe_tokens(engine.active_tokens['ce_token'],
engine.active_tokens['pe_token'])
                        engine.active_tokens.update({"atm": atm, "ce_sym": ce_sym, "ce_token":
ce_token, "pe_sym": pe_sym, "pe_token": pe_token})
                        subscribe_tokens(ce_token, pe_token)
                        send_alert(f"🎯 <b>Strikes Shifted:</b>\nCE: {ce_sym}\nPE: {pe_sym}",
urgent=False)
```

```
                idx_seed = await asyncio.to_thread(engine.fetch_rest_seed, INDEX_TOKEN,
"NSE")
                ce_seed = await asyncio.to_thread(engine.fetch_rest_seed, ce_token, "NFO")
                pe_seed = await asyncio.to_thread(engine.fetch_rest_seed, pe_token, "NFO")
                engine.idx_ind.seed_from_rest(idx_seed)
                engine.ce_ind.seed_from_rest(ce_seed)
                engine.pe_ind.seed_from_rest(pe_seed)

        # 4. Watchdog & Reconnection
        if current_time >= datetime.time(9, 15) and (time.time() - last_tick_time > 5.0) and not
is_reconnecting:
            is_reconnecting = True
            send_alert("⚠️ <b>Watchdog:</b> Stale socket. Reconnecting...", urgent=True)
            if ws_instance:
                try: ws_instance.close()
                except Exception: pass
            await asyncio.sleep(2)

            if api and session_data:
                ws_instance = SmartWebSocketV2(session_data['jwtToken'], API_KEY, CLIENT_ID,
api.getfeedToken())
                ws_instance.on_data = on_data
                ws_instance.on_open = on_open
                ws_instance.on_error = on_error
                ws_instance.on_close = on_close
                threading.Thread(target=ws_instance.connect, daemon=True).start()
                last_tick_time = time.time()

                if engine.trade_state["active"]:
                    subscribe_tokens(engine.trade_state["token"], None)
                elif engine.active_tokens['ce_token']:
                    subscribe_tokens(engine.active_tokens['ce_token'],
engine.active_tokens['pe_token'])

                idx_seed = await asyncio.to_thread(engine.fetch_rest_seed, INDEX_TOKEN, "NSE")
                engine.idx_ind.seed_from_rest(idx_seed)
            is_reconnecting = False

        # 5. Terminal Square-Off (15:26)
        if current_time >= datetime.time(15, 26) and current_time < datetime.time(15, 27):
            if engine and engine.trade_state['active']:
                token = engine.trade_state['token']
                ltp = live_prices.get(token, {}).get('ltp', 0)
```

```
            await engine.exit_trade(ltp, "15:26 Hard Square-Off")

        if engine and engine.trade_log:
            wins = sum(1 for t in engine.trade_log if t['pnl'] > 0)
            send_alert(f"🛑 <b>EOD REPORT</b>\nTotal Legs: {len(engine.trade_log)}\nWin
Rate: {(wins/len(engine.trade_log))*100:.1f}%\nNet PnL: ₹{engine.trade_state['daily_pnl']:.2f}",
urgent=True)
            engine.trade_log.clear()
        await asyncio.sleep(60)
        continue

    # 6. Live Execution Loop (09:15 - 15:15 Freeze)
    if engine and datetime.time(9, 15) <= current_time < datetime.time(15, 15):
        idx = live_prices.get(INDEX_TOKEN)
        ce = live_prices.get(engine.active_tokens['ce_token'])
        pe = live_prices.get(engine.active_tokens['pe_token'])

        if idx: engine.idx_ind.update_tick(idx['ltp'], idx['vol'], idx['time'])
        if ce: engine.ce_ind.update_tick(ce['ltp'], ce['vol'], ce['time'])
        if pe: engine.pe_ind.update_tick(pe['ltp'], pe['vol'], pe['time'])

        # Evaluate Circuit Breakers
        breaker_hit = (engine.trade_state['daily_pnl'] <= MAX_DAILY_LOSS) or
(engine.trade_state['trades_today'] >= MAX_DAILY_TRADES)

        if engine.trade_state['active']:
            # Always manage open trades regardless of circuit breaker state
            opt = live_prices.get(engine.trade_state['token'])
            if opt and idx:
                await engine.check_exits(opt['ltp'], idx['ltp'])

        elif not breaker_hit and idx and engine.idx_ind.prev_close:
            # Check CE (Calls)
            if ce and ce['ltp'] > engine.ce_ind.vwap and engine.ce_ind.prev_close:
                poc_ce = engine.ce_ind.get_poc()
                ema_bullish = (engine.ce_ind.ema9 is not None) and (engine.ce_ind.ema9 >
engine.ce_ind.ema15)
                rg_divergence = (idx['ltp'] < engine.idx_ind.prev_close) and (ce['ltp'] >
engine.ce_ind.prev_close)
                if ema_bullish and rg_divergence and poc_ce and (ce['ltp'] > poc_ce + 1.5):
                    await engine.execute_trade(
                        "CE", engine.active_tokens['ce_token'], engine.active_tokens['ce_sym'],
                        ce['ltp'], idx['ltp'], engine.ce_ind.atr14, engine.idx_ind.atr14
                    )
```

```
            # Check PE (Puts)
            elif pe and pe['ltp'] > engine.pe_ind.vwap and engine.pe_ind.prev_close and not
engine.trade_state['active']:
                poc_pe = engine.pe_ind.get_poc()
                ema_bullish = (engine.pe_ind.ema9 is not None) and (engine.pe_ind.ema9 >
engine.pe_ind.ema15)
                gg_divergence = (idx['ltp'] > engine.idx_ind.prev_close) and (pe['ltp'] >
engine.pe_ind.prev_close)
                if ema_bullish and gg_divergence and poc_pe and (pe['ltp'] > poc_pe + 1.5):
                    await engine.execute_trade(
                        "PE", engine.active_tokens['pe_token'], engine.active_tokens['pe_sym'],
                        pe['ltp'], idx['ltp'], engine.pe_ind.atr14, engine.idx_ind.atr14
                    )

        await asyncio.sleep(0.01)

if __name__ == "__main__":
    asyncio.run(main_loop())
```